

Classical Machine Learning

AI Bootcamp in Biotechnology – Rutgers and KAIMRC Academy

W. Evan Johnson, Ph.D.

Professor, Division of Infectious Disease

Director, Center for Data Science

Associate Director, Center for Biomedical Informatics and Health AI

Rutgers University – New Jersey Medical School

2026-09-15

Support Vector Machines

Support Vector Machines

Support vector machines or **SVMs** are supervised learning models with associated learning algorithms that analyze data used for classification and regression analysis.

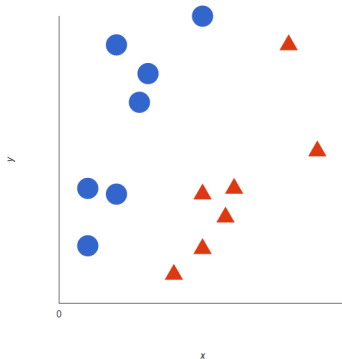
The goal is to find a classifier from an optimized *decision boundary* or “*separating hyperplane*” between two classes.

Material for this lecture was obtained and adapted from:

- ▶ <https://www.datacamp.com/community/tutorials/support-vector-machines-r>
- ▶ *The Elements of Statistical Learning*, Hastie, et al., Springer
- ▶ <https://www.geeksforgeeks.org/classifying-data-using-support-vector-machinessvm-in-r/>

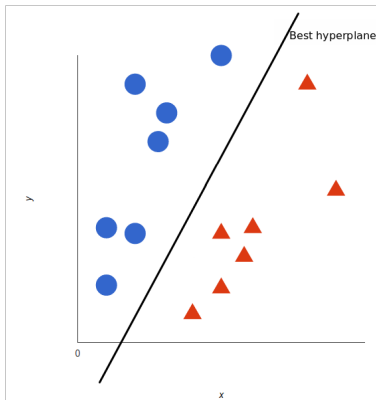
Support Vector Machines—Linear Data

Let's imagine we have two tags: *red* and *blue*, and our data has two features: x and y . We can plot our training data on a plane:



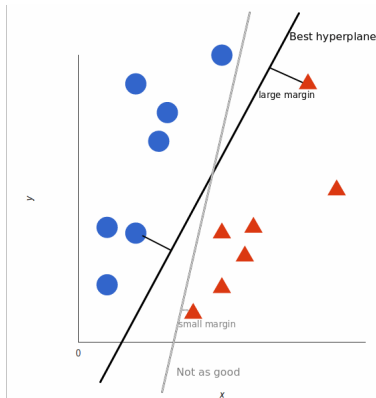
Support Vector Machines

An **SVM** identifies the **decision boundary** or **hyperplane** (two dimensions: line) that best separates the tags:



Support Vector Machines

But, what exactly is the best hyperplane? For SVM, it's the one that maximizes the margins from the data from both tags:

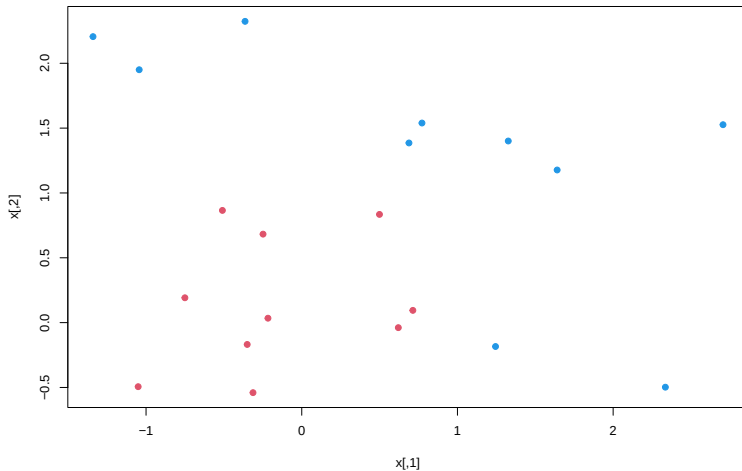


Support Vector Machines in R

First generate some data in 2 dimensions, and make them a little separated:

```
set.seed(10111)
x = matrix(rnorm(40), 20, 2)
y = rep(c(-1, 1), c(10, 10))
x[y == 1,] = x[y == 1,] + 1
plot(x, col = y + 3, pch = 19)
```

Support Vector Machines in R



Support Vector Machines in R

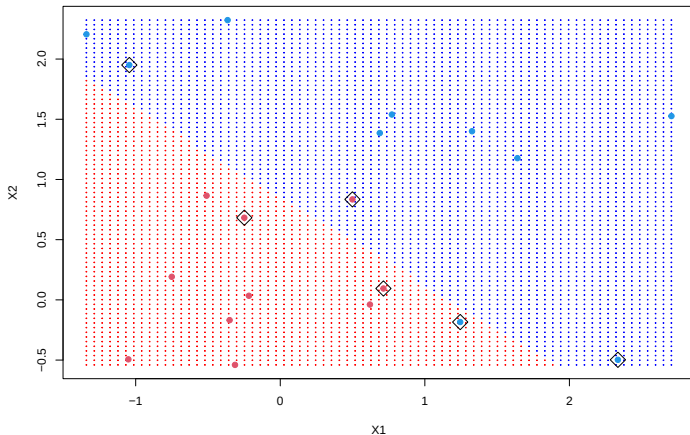
We will use the **e1071** package which contains the `svm` function and make a dataframe of the data, turning `y` into a factor variable.

```
library(e1071)
dat = data.frame(x, y = as.factor(y))
svmfit = svm(y ~ ., data = dat, kernel = "linear", cost = 10, scale = FALSE)
print(svmfit)
```

```
##
## Call:
## svm(formula = y ~ ., data = dat, kernel = "linear", cost = 10, scale = FALSE)
##
##
## Parameters:
##   SVM-Type:  C-classification
##   SVM-Kernel: linear
##           cost: 10
##
## Number of Support Vectors:  6
```

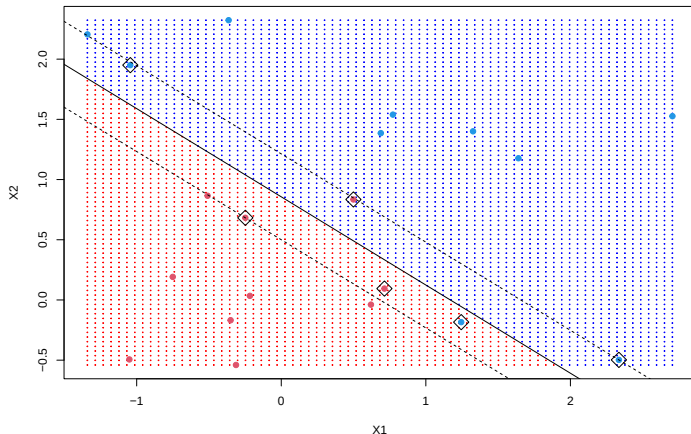
Support Vector Machines in R

Or plotting it more cleanly:



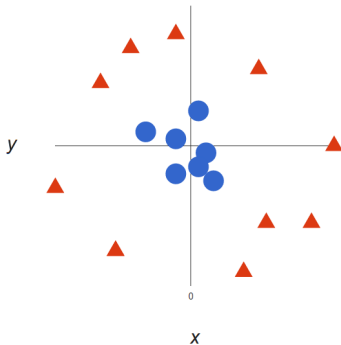
Support Vector Machines in R

Now plotting the lines on the graph:



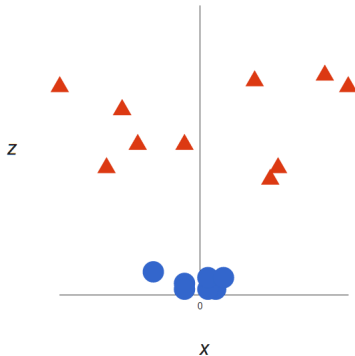
Support Vector Machines–Non-Linear Data

The prior examples were easy because the data were linearly separable. Often things aren't that simple. Look at this case:



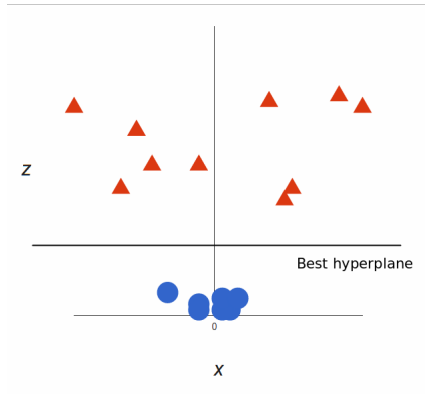
Support Vector Machines

Here we will add a third dimension: $z = x^2 + y^2$ (you'll notice that's the equation for a circle!), and plot x and z .



Support Vector Machines

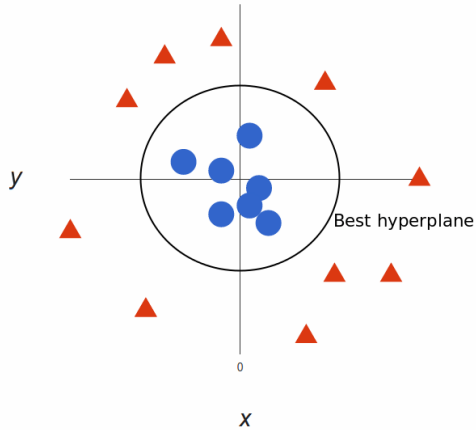
What can SVM do with this? Let's see:



That's great! Note that since we are in three dimensions now, the hyperplane is a

Support Vector Machines

What's left is mapping it back to two dimensions:



The “Kernel Trick”

Transformations are computationally expensive, but SVM doesn't need the actual vectors, it only needs the dot products.

We can tell SVM to do its thing using the new dot product—we call this a **kernel function**.

The “Kernel Trick”

This often called the “*kernel trick*”, which enlarges the feature space for a non-linear boundary between the classes.

Common types of kernels: *linear*, *polynomial*, and *radial basis* kernels.

Simply, these kernels transform our data to pass a linear hyperplane and thus classify our data.

Support Vector Machines in R: Non-linear SVM

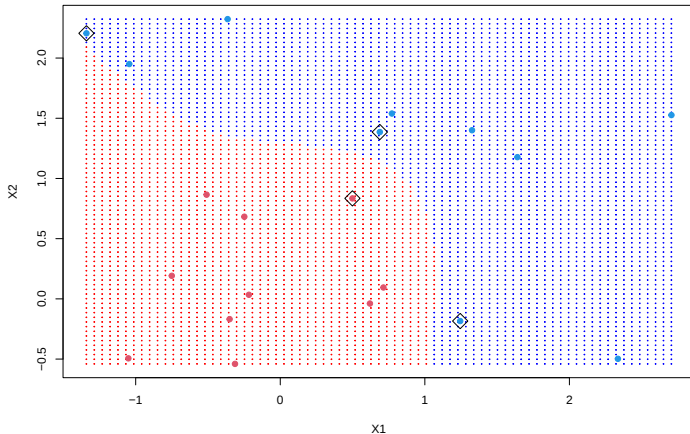
Now let's apply a non-linear (polynomial) SVM to our prior simulated dataset.

```
dat = data.frame(x, y = as.factor(y))
svmfit = svm(y ~ ., data = dat, kernel = "polynomial", cost = 10, scale = FALSE)
print(svmfit)
```

```
##
## Call:
## svm(formula = y ~ ., data = dat, kernel = "polynomial", cost = 10,
##      scale = FALSE)
##
##
## Parameters:
##   SVM-Type:  C-classification
##   SVM-Kernel: polynomial
##         cost: 10
##        degree: 3
##       coef.0: 0
##
## Number of Support Vectors: 4
```

Support Vector Machines in R: Non-linear SVM

Plotting the result:



Support Vector Machines in R: Non-linear SVM

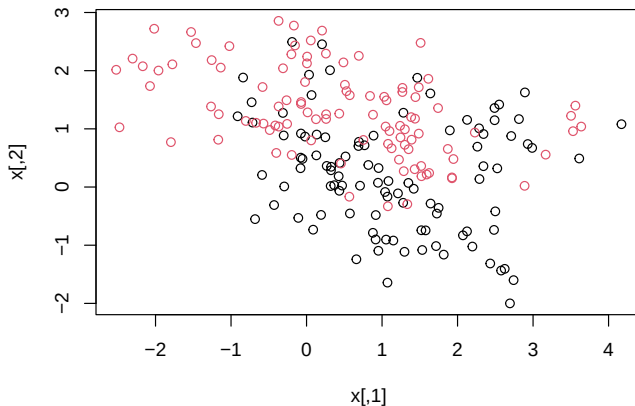
Here is a more complex example from *Elements of Statistical Learning*, where the decision boundary needs to be non-linear and there is no clear separation.

```
#download.file(  
#  "http://www-stat.stanford.edu/~tibs/ElemStatLearn/datasets/ESL.mixture.rda",  
#  destfile='ESL.mixture.rda')  
rm(x,y)  
load(file = "ESL.mixture.rda")  
attach(ESL.mixture)  
names(ESL.mixture)
```

```
## [1] "x"      "y"      "xnew"   "prob"   "marginal" "px1"    "px2"  
## [8] "means"
```

Support Vector Machines in R: Non-linear SVM

Plotting the data:



Support Vector Machines in R: Non-linear SVM

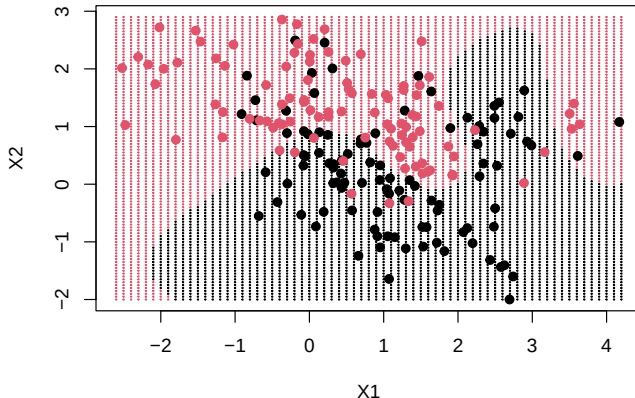
Now make a data frame with the response y , and turn that into a factor. We will fit an SVM with radial kernel.

```
dat = data.frame(y = factor(y), x)
fit = svm(factor(y) ~ ., data = dat, scale = FALSE, kernel = "radial", cost = 5)
print(fit)
```

```
##
## Call:
## svm(formula = factor(y) ~ ., data = dat, kernel = "radial", cost = 5,
##      scale = FALSE)
##
##
## Parameters:
##   SVM-Type:  C-classification
##   SVM-Kernel: radial
##           cost: 5
##
## Number of Support Vectors: 103
```

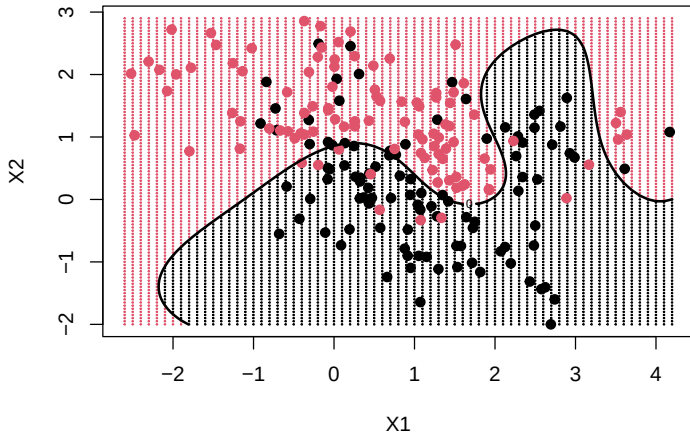
Support Vector Machines in R: Non-linear SVM

It's time to create a grid and predictions. We use `expand.grid` to create the grid, predict each of the values on the grid, and plot them:



Support Vector Machines in R: Non-linear SVM

Plotting with a contour:



Advantages and Disadvantages of SVMs

Advantages:

- ▶ **High Dimensionality:** SVM is an effective tool in spaces where dimensionality is large.
- ▶ **Memory Efficiency:** Only a subset of the training points are used, so just these points need to be stored in memory.
- ▶ **Versatility:** Class separation is often highly non-linear. The ability to apply new kernels allows flexibility for the decision boundaries.

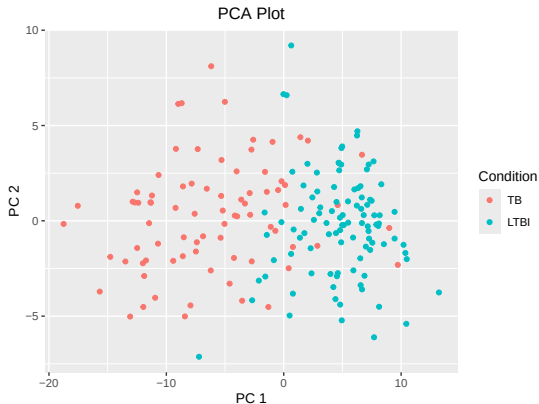
Advantages and Disadvantages of SVMs

Disadvantages:

- ▶ **Kernel Selection:** SVMs are very sensitive to the choice of the kernel parameters.
- ▶ **High dimensionality:** When the number of features for each object exceeds the number of training data samples, SVMs can perform poorly.
- ▶ **Non-Probabilistic:** The classifier works by placing objects above/below a classifying hyperplane, there is no direct probabilistic interpretation.

SVM Nanostring

Remember the PCA dimension reduction of the TB Nanostring dataset. The points are colored based on TB status.



SVM Nanostring

Now try an SVM on the PCs of the Nanostring data:

```
# use only the first 10 PCs
dat = data.frame(y = pca_reduction$Condition, pca_reduction[,1:2])
fit = svm(y ~ ., data = dat, scale = FALSE, kernel = "linear", cost = 10)
print(fit)

##
## Call:
## svm(formula = y ~ ., data = dat, kernel = "linear", cost = 10, scale = FALSE)
##
##
## Parameters:
##   SVM-Type:  C-classification
##   SVM-Kernel: linear
##         cost:  10
##
## Number of Support Vectors:  52
```

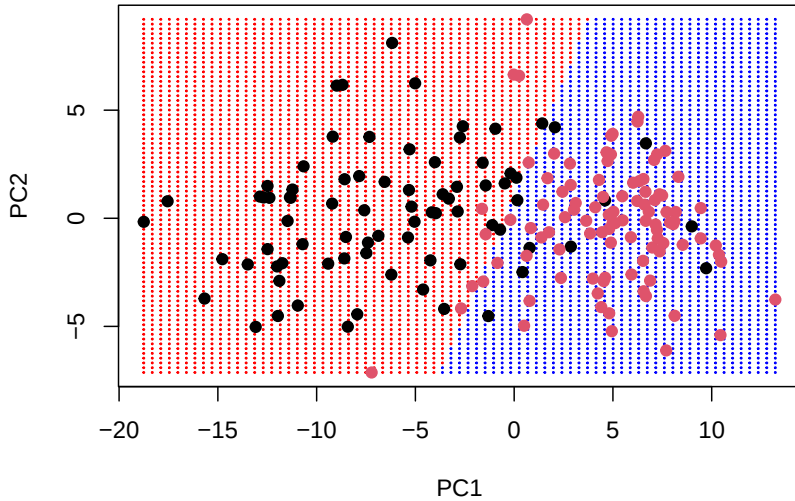
SVM Nanostring

We can evaluate the predictor with a Confusion matrix:

```
library(caret)
confusionMatrix(dat$y,predict(fit,dat))
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction TB LTBI
##      TB    69    10
##      LTBI   8    92
##
##              Accuracy : 0.8994
##              95% CI : (0.8457, 0.9393)
##      No Information Rate : 0.5698
##      P-Value [Acc > NIR] : <2e-16
##
##              Kappa : 0.7955
##
##  Mcnemar's Test P-Value : 0.8137
##
##              Sensitivity : 0.8961
##              Specificity : 0.9020
##      Pos Pred Value : 0.8734
##      Neg Pred Value : 0.9200
##      Prevalence : 0.4302
```

SVM Nanostring



Regularization: Ridge, LASSO, and Elastic Net

Introduction to Regression

The most common form of regression analysis is **linear regression**, in which one finds the line (or a more complex linear combination) that most closely fits the data according to a specific mathematical criterion.

Introduction to Regression

In your math classes, you used the following model for a line, where m represents the slope and b represents the y -intercept:

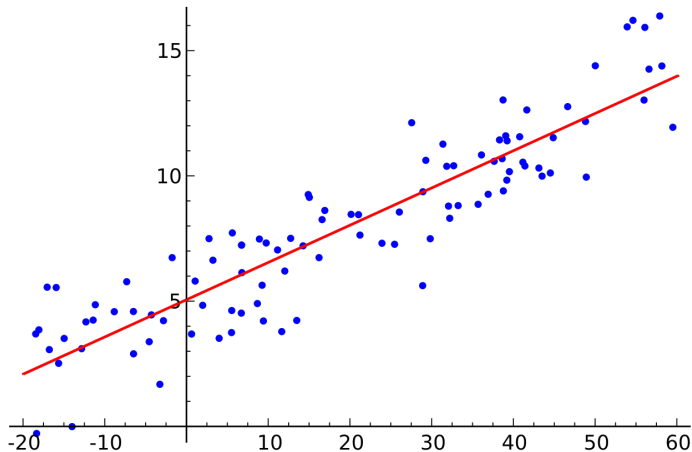
$$y = mx + b.$$

In statistics, we use a slightly different formulation and notation:

$$y_i = \beta_0 + x_i\beta_1,$$

which describes a line with slope β_1 and y -intercept β_0 , and the y_i and x_i represent the observed points from your dataset.

Introduction to Regression



Introduction to Regression

Formal Definition: suppose we observe n data pairs and call them

$$(x_i, y_i), i = 1, \dots, n.$$

We can describe the underlying relationship between y_i and x_i involving this error term ϵ_i by

$$y_i = \beta_0 + x_i\beta_1 + \epsilon_i.$$

We call the deviations of the data from the line the **errors**. It is also important to note that the true underlying parameters β_0 and β_1 are not observed, but must be *estimated* using the data.

Introduction to Regression

The goal is to find estimated values $\hat{\beta}_0$ and $\hat{\beta}_1$ for the parameters β_0 and β_1 which would provide the “best fit” in some sense for the data points.

Here, the “best fit” will be understood as in the **least-squares** approach: a line that minimizes the sum of squared residuals

$$\min_{\beta_0, \beta_1} Q(\beta_0, \beta_1) = \inf_{\beta_0, \beta_1} \left\{ \sum_{i=1}^n (y_i - \beta_0 - x_i \beta_1)^2 \right\} = \inf_{\beta_0, \beta_1} \sum_{i=1}^n \epsilon_i^2.$$

Ridge regression

Ridge regression **regularizes** (shrinks) coefficients by imposing a penalty on their size. The ridge coefficients minimize a penalized sum of squared error:

$$\hat{\beta}^{ridge} = \inf_{\beta} \left\{ \sum_{i=1}^N (y_i - \sum_{j=1}^p x_{ij} \beta_j)^2 + \lambda \sum_{j=1}^p \beta_j^2 \right\},$$

where $\lambda \geq 0$ is a parameter that controls the shrinkage. The larger the value of λ the more shrinkage (towards 0).

Lasso regression

Lasso (Least Absolute Shrinkage and Selection Operator) regression also **regularizes** coefficients by imposing a penalty on their size, but it uses an **L1** penalty. The lasso coefficients minimize the following cost function:

$$\hat{\beta}^{lasso} = \inf_{\beta} \left\{ \sum_{i=1}^N (y_i - \sum_{j=1}^p x_{ij} \beta_j)^2 + \alpha \sum_{j=1}^p |\beta_j| \right\},$$

where $\alpha \geq 0$ is a parameter that controls the shrinkage. The larger the value of α the more shrinkage (towards 0).

Lasso vs Ridge regression

S.No	L1 Regularization	L2 Regularization
1	Panelizes the sum of absolute value of weights.	penalizes the sum of square weights.
2	It has a sparse solution.	It has a non-sparse solution.
3	It gives multiple solutions.	It has only one solution.
4	Constructed in feature selection.	No feature selection.
5	Robust to outliers.	Not robust to outliers.
6	It generates simple and interpretable models.	It gives more accurate predictions when the output variable is the function of whole input variables.
7	Unable to learn complex data patterns.	Able to learn complex data patterns.
8	Computationally inefficient over non-sparse conditions.	Computationally efficient because of having analytical solutions.

(<https://www.analyticssteps.com/blogs/l2-and-l1-regularization-machine-learning>)

Elastic net regularization

Which should I choose? Ridge or Lasso? Well, why do I have to choose! Instead use the **Elastic Net** that minimizes:

$$\hat{\beta}^{\text{elastic net}} = \inf_{\beta} \left\{ \sum_{i=1}^N (y_i - \sum_{j=1}^p x_{ij} \beta_j)^2 + \alpha \sum_{j=1}^p |\beta_j| + \lambda \sum_{j=1}^p \beta_j^2 \right\},$$

for some $\alpha \geq 0$ and $\lambda \geq 0$.

The quadratic penalty term makes the loss function strongly convex, and it therefore has a unique minimum. The elastic net method includes OLS, Lasso, and Ridge regression by setting either $\alpha = 0$, $\lambda = 0$, or both to 0.

Regression Regularization in R: glmnet

We can use the **glmnet** package to apply regularization in R:

```
install.packages("glmnet")
```

Regression Regularization in R: glmnet

Using the quick start example from the package:

```
library(glmnet)
data(QuickStartExample)
x <- QuickStartExample$x
y <- QuickStartExample$y
```

Example: Logistic Regression Elastic Net

Logistic regression is a widely-used model when the response is binary. Suppose the response variable y takes values $\{0,1\}$. We model

$$P(y = 1|\mathbf{X}) = \frac{e^{\mathbf{X}\beta}}{1 + e^{\mathbf{X}\beta}},$$

which can be written in the following form:

$$\log \frac{P(y = 1|\mathbf{X})}{P(y = 0|\mathbf{X})} = \mathbf{X}\beta,$$

the so-called “logistic” or log-odds transformation.

Example: Logistic Regression Elastic Net

Using the example dataset from the glmnet package:

```
library(glmnet)
data(BinomialExample)
x <- BinomialExample$x
y <- BinomialExample$y
```

Example: Logistic Regression Elastic Net

Set family option to “binomial” in the glmnet function:

```
fit <- glmnet(x, y, family = "binomial")
```

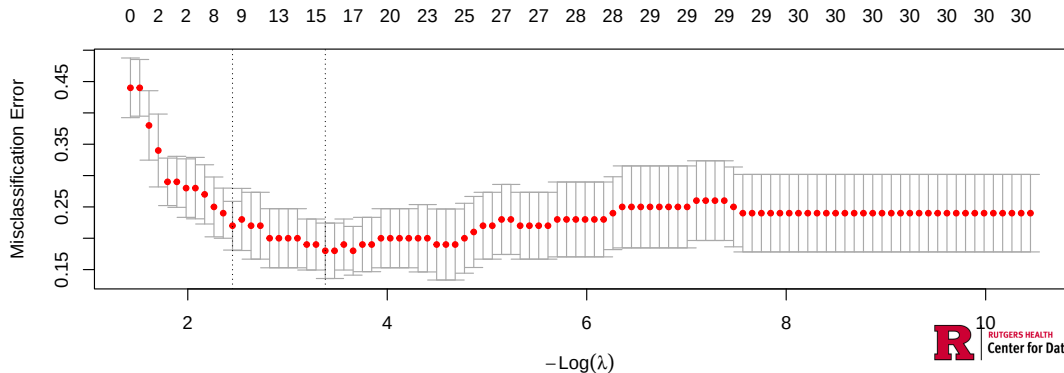
The code below uses misclassification error as the criterion for 10-fold cross-validation:

```
cvfit <- cv.glmnet(x, y, family = "binomial",  
                  type.measure = "class")
```

Example: Logistic Regression Elastic Net

Now we can plot the cross-validation results and find the 'best' λ_{min} :

```
plot(cvfit)
```



Example: Logistic Regression Elastic Net

```
coef(cvfit, s = "lambda.min")
```

```
## 31 x 1 sparse Matrix of class "dgCMatrix"
##          lambda.min
## (Intercept) 0.21936669
## V1          .
## V2          0.37862901
## V3         -0.28601015
## V4         -0.81387974
## V5         -0.08536167
## V6         -0.58258679
## V7          .
## V8         -0.31550430
## V9          0.39716469
## V10        -0.93247315
## V11         .
## V12         .
## V13         .
## V14         .
## V15         .
## V16         0.04921999
## V17         .
## V18         .
## V19         .
## V20         .
## V21         .
## V22         .
```

Decision Trees

Motivating example: olive oil

To motivate this section, we will use a new dataset that includes the breakdown of the composition of olive oil into 8 fatty acids:

```
library(tidyverse)
library(dslabs)
data("olive")
olive <- select(olive, -area) #remove the `area` column--don't use it
names(olive)
```

```
## [1] "region"      "palmitic"    "palmitoleic" "stearic"     "oleic"
## [6] "linoleic"    "linolenic"   "arachidic"   "eicosenoic"
```

Motivating example: olive oil

Here is a quick look at the data:

```
olive %>% head() %>%  
  knitr::kable() %>% kable_styling(font_size =7)
```

region	palmitic	palmitoleic	stearic	oleic	linoleic	linolenic	arachidic	eicosenoic
Southern Italy	10.75	0.75	2.26	78.23	6.72	0.36	0.60	0.29
Southern Italy	10.88	0.73	2.24	77.09	7.81	0.31	0.61	0.29
Southern Italy	9.11	0.54	2.46	81.13	5.49	0.31	0.63	0.29
Southern Italy	9.66	0.57	2.40	79.52	6.19	0.50	0.78	0.35
Southern Italy	10.51	0.67	2.59	77.71	6.72	0.50	0.80	0.46
Southern Italy	9.11	0.49	2.68	79.24	6.78	0.51	0.70	0.44

Motivating example: olive oil

We will try to predict the region using the fatty acid composition values as predictors.

```
table(olive$region)
```

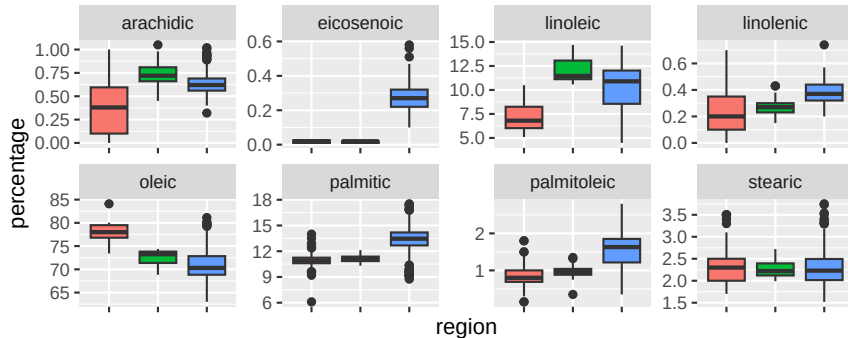
```
##
```

```
## Northern Italy          Sardinia Southern Italy
```

```
##           151              98              323
```

Motivating example: olive oil

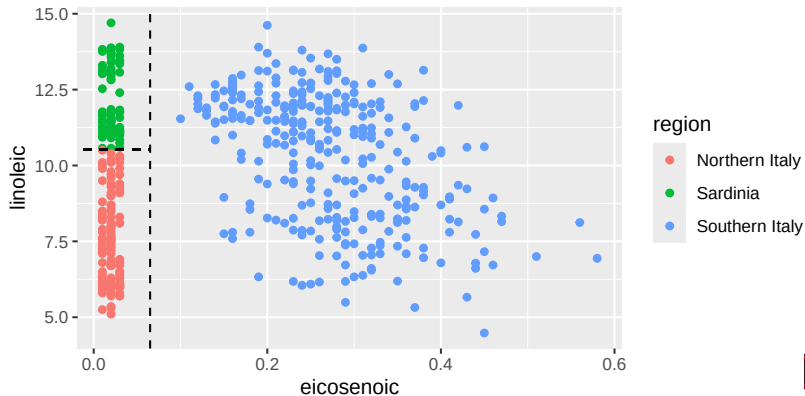
A bit of data exploration reveals that the regions should be separable: note that eicosenoic is only in Southern Italy and linoleic separates Northern Italy from Sardinia.



region  Northern Italy  Sardinia  Southern Italy

Motivating example: olive oil

This implies that we should be able to build an algorithm that predicts perfectly! Let's try plotting the values for eicosenoic and linoleic.



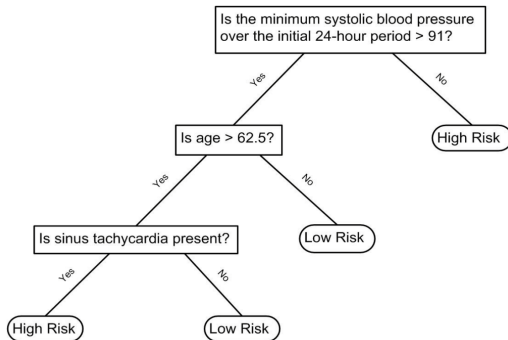
Motivating example: olive oil

Let's define a **decision rule**: If eicosenoic is larger than 0.065, predict Southern Italy. If not, then if linoleic is larger than 10.535, predict Sardinia, otherwise predict Northern Italy. We can draw this decision tree:



Decision Trees

Decision trees like this are often used in practice. For example, to evaluate a person's risk of poor outcome after a heart attack, doctors may use:



(Source: Walton 2010 Informal Logic, Vol. 30, No. 2, pp. 159-184.)

Decision Trees

A tree is a flow chart of yes or no questions. We define an algorithm that creates trees with predictions at the ends, referred to as **nodes**.

Regression and decision trees, or sometimes called **Classification and Regression Trees (CART)**, predict an outcome Y by partitioning the predictors X .

Classification (decision) trees

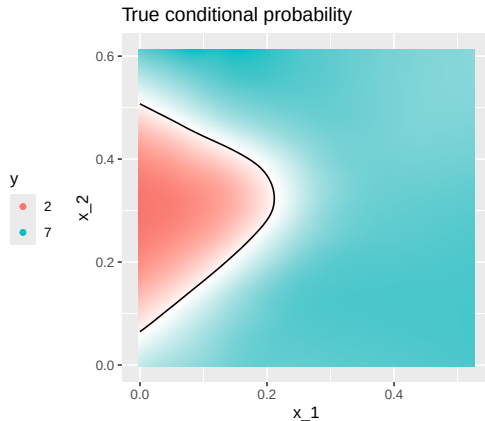
Classification trees predict a **categorical** outcome.

The tree splits the predictors into partitions, then:

- ▶ Each partition predicts the **most common class** in it
- ▶ Splits are chosen to make partitions as **pure** as possible, measured by the **Gini index** or **entropy**

Classification (decision) trees

Lets look at how a classification tree performs on the following dataset:



Classification (decision) trees

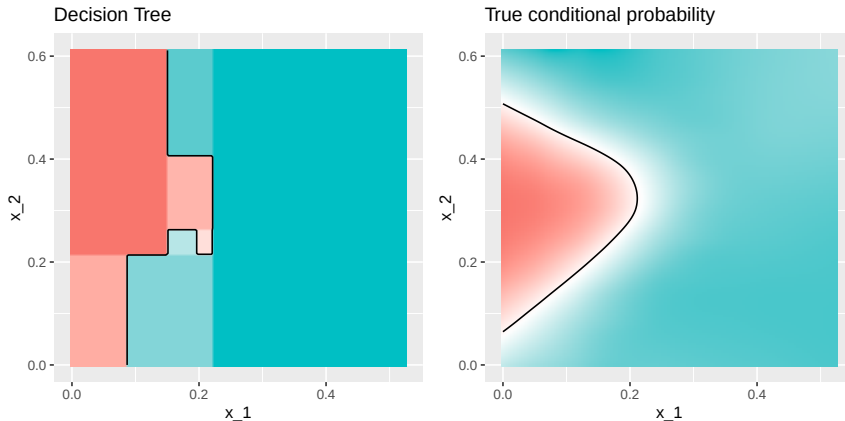
The accuracy achieved by this approach is better than what we got with regression, but is not as good as what we achieved with kernel methods:

```
y_hat <- predict(train_rpart, mnist_27$test)
confusionMatrix(y_hat, mnist_27$test$y)$overall["Accuracy"]
```

```
## Accuracy
##      0.81
```

Classification (decision) trees

The conditional probability shows us the limitations of classification trees:



Classification (decision) trees

Classification trees have certain advantages that make them very useful.

- ▶ They are highly interpretable, even more so than linear models.
- ▶ They are easy to visualize (if small enough).
- ▶ They can model human decision processes and don't require use of dummy predictors for categorical variables.

On the other hand:

- ▶ The approach via recursive partitioning can easily over-train and is therefore a bit harder to train than, for example, linear regression or kNN.
- ▶ In terms of accuracy, it is rarely the best performing method since it is not very flexible and is highly unstable to changes in training data.

The Problem: High Variance and Instability

A major drawback of a single decision tree is its **high variance** and **instability**. Small changes in the training data can lead to a completely different tree structure.

- ▶ **High Variance:** Because the tree structure is based on local optimal splits, a slight perturbation in the data can drastically change the first split, cascading errors throughout the entire tree.
- ▶ **Instability:** A single tree is not robust. If you fit two trees on two slightly different subsets of the data (e.g., train/test splits), they will likely produce very different decision boundaries.

Ensembles: Bagging and Boosting

Ensemble Methods: Bagging vs. Boosting

Ensemble methods combine the predictions of multiple individual models (often called **weak learners**, like decision trees) to create a single, highly accurate predictor.

The two main strategies for building ensembles are:

- ▶ **Bagging (Bootstrap Aggregation)**: A *parallel* method where models are built independently and then averaged/combined. Used in **Random Forests**.
- ▶ **Boosting**: A *sequential* method where models are built iteratively, with each new model focusing on correcting the errors of the previous ones. Used in **XGBoost** (introduced soon).

Bagging (Bootstrap Aggregation)

Bagging is a technique designed primarily to **reduce variance** in high-variance models, such as deep decision trees.

1. **Parallelism:** B independent models are trained simultaneously.
2. **Bootstrap Sampling:** Each model is trained on a unique *bootstrap sample* (sampling with replacement) of the original training data. This ensures the individual trees are diverse (random).
3. **Final Prediction:** The predictions from all B models are combined by:
 - ▶ **Averaging** (for regression, e.g., Random Forest).
 - ▶ **Majority Vote** (for classification, e.g., Random Forest).

The final averaged model has a significantly lower variance than any single tree, but the bias remains roughly the same.

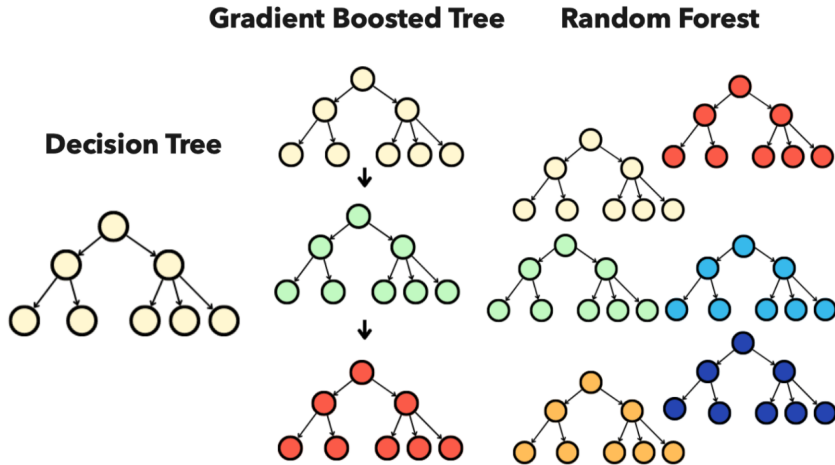
Boosting (Sequential Error Correction)

Boosting is an iterative technique designed primarily to *reduce bias* and create a single strong learner from a sequence of weak learners.

1. **Sequential Dependence:** Models are trained *one after the other*.
2. **Error Focus:** Each subsequent model is trained to predict the *residuals* (the errors) of the combined ensemble created up to that point.
3. **Weighted Combination:** New models are added to the ensemble using a small *learning rate* (η) to ensure the model learns slowly and prevents dramatic overfitting.

Boosting can achieve very high predictive accuracy but requires careful tuning to avoid overfitting the training data, as seen with the complexity parameters in XGBoost.

Ensemble Methods: Bagging vs. Boosting



Random Forests

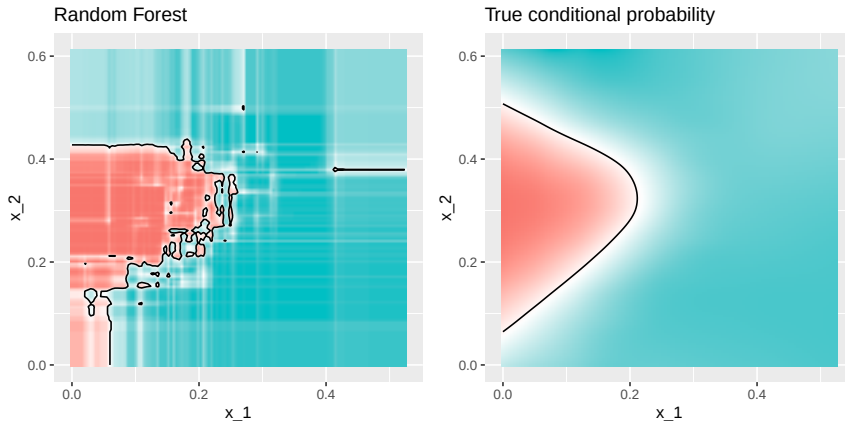
Random forests

Random forests address the shortcomings of decision trees: they improve prediction performance and reduce instability by **averaging** many trees.

The name says how it works – the **bootstrap** makes the individual trees randomly different, and the combination of those trees is the **forest**.

Random forests

Here is what the conditional probabilities look like:



TB Nanostring Example

```
ctrl_cv <- trainControl(method = "repeatedcv",  
                        number = 5, repeats = 3,  
                        classProbs = TRUE,  
                        summaryFunction = twoClassSummary,  
                        savePredictions = "final",  
                        verboseIter = FALSE)
```

TB Nanostring Example

```
set.seed(123)
rf_model <- train(TB_Status ~ ., data = train_data,
  method = "rf", metric = "ROC", importance = TRUE,
  trControl = ctrl_cv)
```

```
# Print optimal hyperparameters found by CV
print(rf_model)
```

```
## Random Forest
##
## 126 samples
## 107 predictors
##   2 classes: 'TB', 'LTBI'
##
## No pre-processing
## Resampling: Cross-Validated (5 fold, repeated 3 times)
## Summary of sample sizes: 101, 100, 101, 101, 101, 101, ...
## Resampling results across tuning parameters:
##
```

TB Nanostring Example

```
rf_pred <- predict(rf_model, test_data)
confusionMatrix(rf_pred, test_data$TB_Status)
```

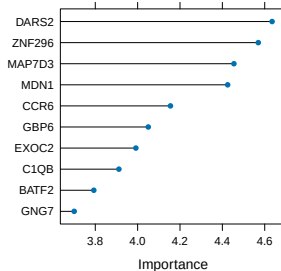
```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction TB LTBI
##      TB      20      4
##      LTBI      3     26
##
##              Accuracy : 0.8679
##              95% CI : (0.7466, 0.9452)
##      No Information Rate : 0.566
##      P-Value [Acc > NIR] : 2.35e-06
##
##              Kappa : 0.7325
##
##      Mcnemar's Test P-Value : 1
##
##              Sensitivity : 0.8606
```

TB Nanostring Example

Variable Importance:

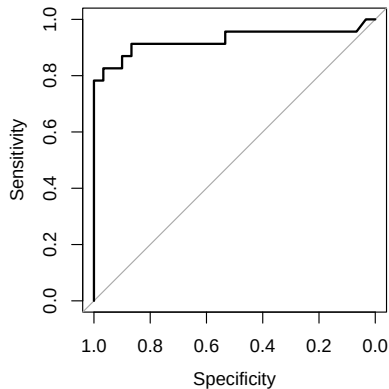
```
rf_imp <- varImp(rf_model, scale = FALSE)
plot(rf_imp, top = 10, main = "Top 10 Important Genes (Random Forest)")
```

Top 10 Important Genes (Random Forest)



TB Nanostring Example

Random Forest ROC Curve (AUC = 0.927)



Gradient Boosting

Introducing Gradient Boosting Machines (GBMs)

Gradient Boosting Machines (GBMs) is the general class of ensemble methods that utilizes the boosting principle.

How it relates to Boosting and XGBoost:

- ▶ **Boosting Foundation:** GBMs use the sequential error correction strategy (Boosting), where new trees correct the mistakes of the previous ensemble.
- ▶ **Gradient Descent:** The "Gradient" in GBM refers to the use of gradient descent—an optimization algorithm—to precisely identify the errors (residuals) and determine the ideal direction for the next tree to move in to minimize the model's overall loss function.
- ▶ **XGBoost is an Implementation:** XGBoost is simply a modern, highly optimized, and regularized (L1/L2) implementation of the GBM algorithm.

GBMs are generally more complex to train than Random Forests but often achieve superior predictive accuracy by aggressively reducing model bias.

Introducing Gradient Boosting Machines (GBMs)



GBM Example: Iris

```
# Load the iris dataset and set up training control  
data("iris")  
set.seed(42) # Set seed for reproducibility  
  
# 5-fold Cross-Validation  
ctrl <- trainControl(method = "cv", number = 5)  
  
# GBM Tuning Grid  
gbmGrid <- expand.grid(n.trees = c(50, 100, 150),  
                      interaction.depth = c(1, 3, 5),  
                      shrinkage = c(0.1),  
                      n.minobsinnode = 10)
```

GBM Example: Iris

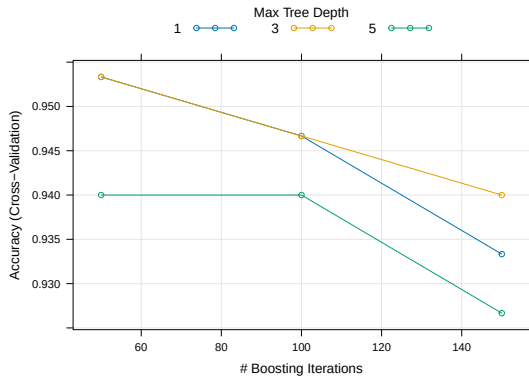
```
# Train the GBM model
gbm_fit <- train(Species ~ ., data = iris,
  method = "gbm", trControl = ctrl,
  verbose = FALSE, tuneGrid = gbmGrid)
```

```
# Print best model results
print(gbm_fit)
```

```
## Stochastic Gradient Boosting
##
## 150 samples
## 4 predictor
## 3 classes: 'setosa', 'versicolor', 'virginica'
##
```

GBM Example: Iris

```
plot(gbm_fit)
```



GBM Example: Iris

```
gbm_predictions <- predict(gbm_fit, newdata = iris)
confusionMatrix(gbm_predictions, iris$Species)
```

```
## Confusion Matrix and Statistics
```

```
##
```

```
##           Reference
```

```
## Prediction   setosa versicolor virginica
```

```
##   setosa      50          0          0
```

```
##   versicolor  0          49          0
```

```
##   virginica   0          1         50
```

```
##
```

```
## Overall Statistics
```

```
##
```

```
##           Accuracy : 0.9933
```

```
##           95% CI : (0.9634, 0.9998)
```

```
##   No Information Rate : 0.3333
```

```
##   P-Value [Acc > NIR] : < 2.2e-16
```

Extreme Gradient Boosting (XGBoost)

XGBoost (Extreme Gradient Boosting) is a highly optimized and scalable implementation of **Gradient Boosting Machines (GBMs)**. It builds upon the idea of decision trees and boosting.

XGBoost: Iris Example

We call `xgboost` directly rather than through `caret`. The `caret` wrapper is a thin shim over a fast-moving API, and it is the part that breaks between versions.

```
data("iris")
set.seed(42)

X      <- as.matrix(iris[, 1:4])
y      <- as.integer(iris$Species) - 1      # xgboost wants 0-based labels
dtrain <- xgb.DMatrix(data = X, label = y)

base_params <- list(objective = "multi:softmax",
                    num_class = 3,
                    eta       = 0.1,          # learning rate
                    eval_metric = "merror")    # classification error
```

XGBoost: Iris Example

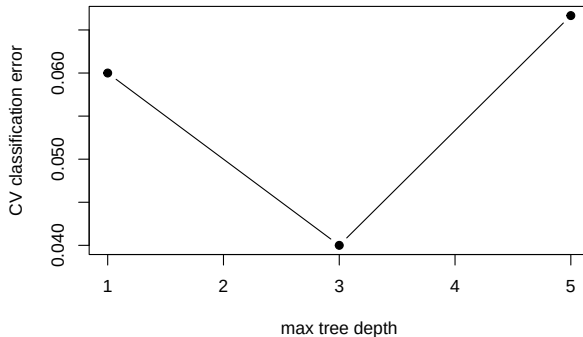
Tune tree depth by 5-fold cross-validation, letting early stopping choose the number of boosting rounds:

```
tune <- do.call(rbind, lapply(c(1, 3, 5), function(d) {  
  cv <- xgb.cv(params = c(base_params, max_depth = d),  
               data    = dtrain,  
               nrounds = 150, nfold = 5,  
               early_stopping_rounds = 20, verbose = 0)  
  log <- as.data.frame(cv$evaluation_log)           # one row per round  
  err <- log[[grep("^test_.*mean$", names(log))[1]]]  
  i   <- which.min(err)  
  data.frame(max_depth = d, nrounds = i, error = err[i])  
}))  
tune
```

##	max_depth	nrounds	error
## 1	1	43	0.06000000
## 2	3	4	0.04000000
## 3	5	6	0.06666667

XGBoost: Iris Example

```
plot(tune$max_depth, tune$error, type = "b", pch = 19,  
     xlab = "max tree depth", ylab = "CV classification error")
```



XGBoost: Iris Example

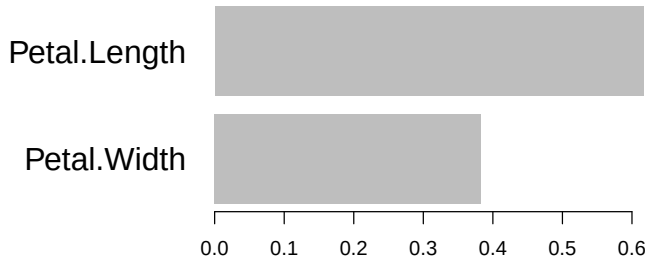
Refit at the best setting:

```
best <- tune[which.min(tune$error), ]  
  
xgb_fit <- xgb.train(params = c(base_params, max_depth = best$max_depth),  
                    data    = dtrain,  
                    nrounds = best$nrounds,  
                    verbose = 0)
```

XGBoost: Iris Example

Which features did the model actually use?

```
xgb.plot.importance(xgb.importance(model = xgb_fit))
```



XGBoost: Iris Example

```
xgb_predictions <- factor(levels(iris$Species)[predict(xgb_fit, dtrain) + 1],  
                           levels = levels(iris$Species))  
confusionMatrix(xgb_predictions, iris$Species)
```

```
## Confusion Matrix and Statistics
```

```
##
```

```
##           Reference
```

```
## Prediction   setosa versicolor virginica
```

```
##   setosa      50          0          0
```

```
##   versicolor   0         47          1
```

```
##   virginica    0          3         49
```

```
##
```

```
## Overall Statistics
```

```
##
```

```
##           Accuracy : 0.9733
```

```
##           95% CI : (0.9331, 0.9927)
```

```
##           No Information Rate : 0.3333
```

Explainability: LIME and SHAP

LIME: Local Interpretable Model-agnostic Explanations

The idea: take one prediction, jiggle the inputs a little, and watch how the prediction moves. Fit a *simple* model to that local neighborhood – its coefficients are the explanation.

- ▶ **Local:** tells you why *this* sample got *this* answer
- ▶ **Model-agnostic:** your model stays a black box
- ▶ Fast, but the explanation depends on how you jiggle

LIME Example: GBM on Iris Data

```
# install.packages("lime")  
library(lime)  
iris_features <- iris[, -5]  
  
# Create a LIME explainer for the GBM model  
gbm_explainer <- lime(  
  x = iris_features,  
  model = gbm_fit  
)
```

LIME Example: GBM on Iris Data

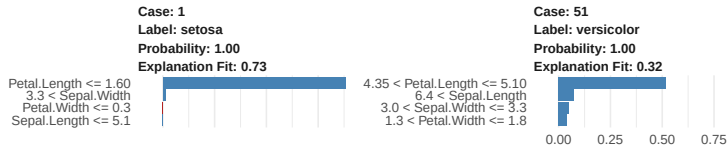
```
# first 2 iris samples
samples_to_explain <- iris_features[c(1,51,101), ]

gbm_lime_results <- explain(
  x = samples_to_explain,
  explainer = gbm_explainer,
  n_features = 4,
  n_labels = 1
)
```

LIME Example: GBM on Iris Data

```
plot_features(gbm_lime_results)
```

```
## Warning: 'aes_()' was deprecated in ggplot2 3.0.0.
## i Please use tidy evaluation idioms with 'aes()'
## i The deprecated feature was likely used in the lime package.
## Please report the issue at <https://github.com/tidymodels/lime/issues>
## This warning is displayed once per session.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```



SHAP: SHapley Additive exPlanations

The idea: borrowed from game theory. Treat the features as players on a team whose payout is the prediction. A feature's SHAP value is its average contribution across every possible team it could have joined.

- ▶ **Additive:** the contributions sum to (this prediction — average prediction)
- ▶ **Consistent:** a feature that matters more never scores lower
- ▶ **Local and global:** aggregate across samples for overall importance

Which one, and when?

	LIME	SHAP
Asks	why did <i>this</i> prediction happen?	how much did each feature contribute?
Basis	a simple local surrogate model	game-theoretic credit assignment
Guarantees	none – depends on the sampling	additive and consistent
Cost	fast	slow, but exact for trees (Tree SHAP)
Scope	local only	local <i>and</i> global

Use LIME to interrogate a single surprising case. Use SHAP when you need the attribution itself to be defensible.

Hands-on practice

Everything in this session has a matching workshop you can work through at your own pace:

`workshop_2`

- ▶ Linear and non-linear support vector machines
- ▶ Decision trees on the olive oil data
- ▶ Random forests on the TB NanoString data

All workshops are in the course repository, with the R Markdown source and a rendered HTML version.

Session Info

```
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.6.2
##
## Matrix products: default
## BLAS: /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/libBLAS.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib; LAPACK version 3.12.1
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: Asia/Qatar
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices utils      datasets  methods   base
##
## other attached packages:
## [1] pROC_1.19.0.1      kableExtra_1.4.0    gridExtra_2.3
## [4] dslabs_0.9.1        lime_0.5.4          xgboost_3.2.1.1
## [7] gbm_2.2.3           randomForest_4.7-1.2 rpart_4.1.27
## [10] e1071_1.7-17        glmnet_5.0           Matrix_1.7-5
## [13] caret_7.0-1         lattice_0.22-9       lubridate_1.9.5
## [16] forcats_1.0.1       stringr_1.6.0        dplyr_1.2.1
## [19] rstan_2.21.0        tidyr_1.2.3
```